

5.7 Industry Core Setup - Example

Version:	1.0.2
Date:	13.12.2024
Status:	Work in Progress / Submitted / Approved
Authors:	Markus Keidl (FG-233) Christopher Winter (FG-233)

Change History

Version	Date	Authors	Changes
1.0.0	26.11.2024	Markus Keidl (FG-233) Christopher Winter (FG-233)	Release of initial version of the Digital Twin Example.
1.0.1	10.12.2024	Markus Keidl (FG-233) Christopher Winter (FG-233)	Add more description for type twins.
1.0.2	13.12.2024	Markus Keidl (FG-233)	Fixed usage of externalSubjectId in digital twin examples.

- [Introduction to the Example](#)
 - [Create DTR Asset](#)
 - [Create dDTR Policies](#)
 - [Access Policy for dDTR](#)
 - [Usage Policy for dDTR](#)
 - [Creating a dDTR Contract Definition](#)
 - [Filter for the dDTR Offer](#)
- [Register Digital Twins in DTR](#)
 - [Registering the Vehicle Digital Twin \(Part Instance\)](#)
 - [Digital Twin Type : Part Type](#)
- [Setup Submodels of Digital Twins in EDC](#)
 - [EDC Setup for Traceability](#)
 - [EDC Setup for Sustainability & Circular Economy](#)
 - [EDC Setup Secondary Material Content](#)
 - [EDC Setup Product Carbon Footprint](#)
 - [Publish Submodels without Asset Bundling \(One EDC Asset per Submodel\)](#)
 - [Publish Submodels with Asset Bundling \(One EDC Asset per Aspect Model\)](#)
- [Create Submodels in the Submodel Server](#)
- [Use Full Definition](#)



The complete example is available for testing in the following BMW EDC:

- EDC URL: <https://connector-partner.edc.aws.bmw.cloud/api/v1/dsp>
- BPN: BPNL0000000J8CWU

Please be aware that every partner that wants to test needs to be added to the EDC access policy BPN group first. For that, write an email with the partner's BPN to [Markus Keidl \(FG-233\)](#) and [Christopher Winter \(FG-233\)](#) .

Additional documentation is available here:

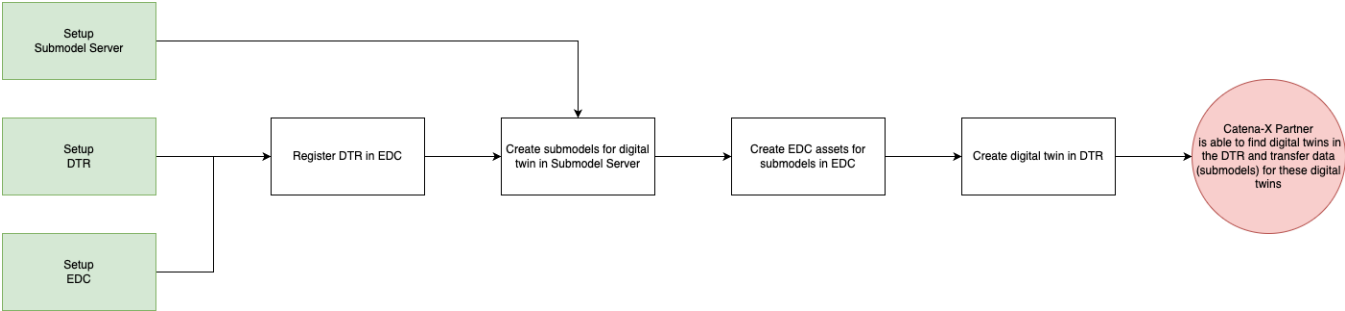
- BMW-internal GitHub repository with all files used in the example: <https://atc-github.azure.cloud.bmw/Extended-Enterprise-Catena-X/catena-x-dt-example>
- Insomnia collection with all request to setup the example or query DTR and submodel data: [Digital-Twin-Example-v1-0.0.json](#)

Introduction to the Example

This example shows you how to create digital twins and provide these in Catana-X in a Industry-Core-compliant way. You need to setup the following components to implement this example:

- Digital Twin Registry (DTR) - to create and publish digital twins
- Submodel Server - to create and provide data (so-called submodels) for digital twins based on aspect models
- EDC - to connect DTR and Submodel Server to Catena-X. Both components, DTR and Submodel Server are only accessible via the EDC.

You need all of these three components and you need to connect them properly to that data consumers can find your digital twins and transfer data (so-called submodels) for these digital twins. The following steps are necessary to successfully setup this example:

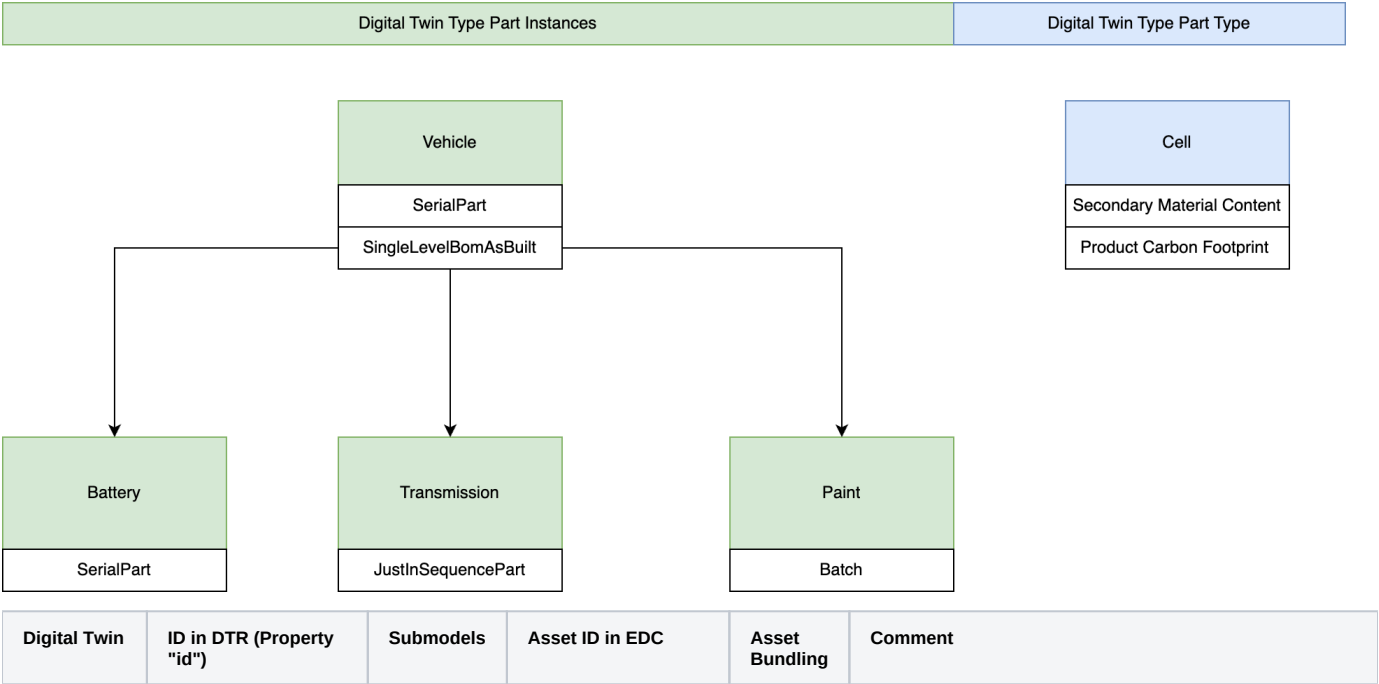


The example consists of

- PartInstance: Vehicle with three components Battery (Serial Part), Transmission (JISPart), Paint (Batch)
 - For the Vehicle digital twin, we create two submodels: SerialPart and SingleLevelBomAsBuilt
 - The BoM of the Vehicle, contains three child parts: a Battery, a Transmission, and Paint. The SingleLevelBomAsBuilt submodel contains this information.
 - For the Battery, Transmission, and Paint digital twins, we only create one submodel: SerialPart, JustInSequencePart, or Batch, depending on the type of part.
 - To publish the data in the EDC we use two approaches which are also described in the Industry Core KIT. More details can be found in the following sections.
 - No asset bundling is used for Vehicle and Transmission digital twins
 - Asset bundling is used for Battery digital twins.
- PartType:
 - 1 Twin for CellType with submodels SRQ, PCF

We also provide an Insomnia collection with all steps necessary to create DTR and EDC artifacts for this example. The Submodel Server uses a simple S3 bucket for data for non-asset-bundling submodels and a simple HTTP server for asset-bundling submodels.

The following figure illustrates the example:



BMW i4 2024-01-01-0001 (Vehicle)	urn:uuid:da071e28-8cf2-46a1-b5af-65231887c7c5				You can use the following query to fetch the digital twin from the DTR (by ID): https://registry-partner.edc.aws.bmw.cloud/api/v3/shell-descriptors/dXJuOnV1aWQ6ZGEwNzFIMjgtOGNmMi00NmExLWl1YWYtNjUyMzE4ODdjN2M1 dXJuOnV1aWQ6ZGEwNzFIMjgtOGNmMi00NmExLWl1YWYtNjUyMzE4ODdjN2M1 is the base64-encoded digital twin ID urn:uuid:da071e28-8cf2-46a1-b5af-65231887c7c5
		SerialPart	Vehicle_2024-01-01-0001_Submodel_SerialPart	No	
		SingleLevelBomAsBuilt	Vehicle_2024-01-01-0001_Submodel_SingleLevelBomAsBuilt	No	
Battery 2024-01-01-0001	urn:uuid:7cf7edda-1efa-459b-b9d7-7e9f29d8d199				
		SerialPart	Battery_Submodel_SerialPart	Yes	
Transmission 2024-01-01-0001	urn:uuid:57d081ea-9d25-4ac5-ada0-7deb0c6b1f07				
		JustInSequencePart	Transmission_2024-01-01-0001_Submodel_JustInSequencePart	No	
Paint					Not yet implemented
LI-ION ZELLE PHEV1 26AH MK02 C2	urn:uuid:5d63f272-8029-4ee6-8e3d-3b1831a30b36				
		PCF	Submodel_Cell_ProductCarbonFootprint		
		SMC	Submodel_Cell_SecondaryMaterialContent		

Setup DTR in EDC

Create DTR Asset

In the initial step, an **EDC asset** must be created for the **decentralized Digital Twin Registry**. This foundational step is necessary to establish a contract definition in the subsequent phase. This lays the groundwork for data consumers to access the decentralized Digital Twin Registry (dDTR) through the EDC. The following JSON can be utilized to create the dDTR asset

dDTR Asset

```
{
  "@context": {
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "cx-common": "https://w3id.org/catenax/ontology/common#",
    "cx-taxo": "https://w3id.org/catenax/taxonomy#",
    "dct": "http://purl.org/dc/terms/"
  },
  "@id": "{{ _ .edcAssetId }}",
  "properties": {
    "dct:type": {
      "@id": "cx-taxo:DigitalTwinRegistry"
    },
    "cx-common:version": "3.0"
  },
  "privateProperties": {
  },
  "dataAddress": {
    "@type": "DataAddress",
    "type": "HttpData",
    "baseUrl": "{{DTR_URL}}",
    "proxyQueryParams": "true",
    "proxyPath": "true",
    "proxyMethod": "false",
    "oauth2:tokenUrl": "{{ _ .url_keycl_backend }}",
    "oauth2:clientId": "{{ _ .client_id_backend }}",
    "oauth2:clientSecretKey": "{{ _ .sec_name_vault }}"
  }
}
```



- **Asset ID** can be freely chosen. We recommend to use a **UUID v4**
- The **dct:type** MUST be set to **cx-taxo:DigitalTwinRegistry**
- The **baseUrl** ends before the /shell-descriptor and /lookup segments.
- The **proxyPath** can be set to true or false. If true the dataplane of the edc will forward the request to the DTR.
- We recommend to set the **proxyMethod** to **false** as CX currently does not foresee to allow 3rd parties to change dDTR entries.

Create dDTR Policies

Access Policy for dDTR

For this example, we will use an access policy that checks if a requestor is an active member of the Catena-X ecosystem and is part of the "industry-core-example" Business Partner Group. Only requestors who are active Catena-X members and have been added to the "industry-core-example" group will be able to view the contract offer. For all other requestors, the offer will not be visible.

The following JSON can be utilized to create access policies.

Access Policy dDTR

```
{
  "@id": "access-policy",
  "@type": "PolicyDefinition",
  "profile": "cx-policy:profile2405",
  "policy": {
    "@type": "odrl:Set",
    "odrl:permission": {
      "odrl:action": {
        "@id": "odrl:use"
      },
      "odrl:constraint": {
        "odrl:and": [
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:Membership"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "active"
          },
          {
            "odrl:leftOperand": {
              "@id": "tx:BusinessPartnerGroup"
            },
            "odrl:operator": {
              "@id": "odrl:isAnyOf"
            },
            "odrl:rightOperand": "industry-core-example"
          }
        ]
      }
    }
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

Usage Policy for dDTR

The usage policy defines the conditions under which data from the decentralized Digital Twin Registry (dDTR) may be utilized. By using a standardized purpose such as "cx.core.digitalTwinRegistry:1", it ensures that all providers within the Catena-X ecosystem employ a consistent usage policy. This enhances the transparency and trustworthiness of the system.

dDTR Usage Policy

```
{
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  },
  "@id": "USEAGEPOLICYIDfordDTR",
  "policy": {
    "@type": "odrl:Set",
    "odrl:permission": {
      "odrl:action": {
        "@id": "odrl:use"
      },
      "odrl:constraint": {
        "odrl:and": [
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:FrameworkAgreement"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "DataExchangeGovernance:1.0"
          },
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:UsagePurpose"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "cx.core.digitalTwinRegistry:1"
          }
        ]
      }
    }
  }
}
```

Creating a dDTR Contract Definition



Info

In common a contract definition always consists out of 1 to n assets an access policy and 1 to n usage policies. So you should first create a access and usage policy for your registry (like shown above). In common policies can be reused, but please **do not reuse the usage policy** of the registry for other contract definitions as the purpose of the usage is focused on the dDTR use!

dDTR contract definition

```
{
  "@id": "Contract-Definition-Digital-Twin-Registry",
  "@type": "ContractDefinition",
  "accessPolicyId": "access-policy",
  "contractPolicyId": "usage-policy-for-digitalTwinRegistry",
  "assetsSelector": {
    "@type": "Criterion",
    "operandLeft": "http://purl.org/dc/terms/type",
    "operator": "like",
    "operandRight": "%https://w3id.org/catenax/taxonomy#DigitalTwinRegistry%"
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

Filter for the dDTR Offer

If everything is setup like described above, a data consumer should find the **dDTR offer** in the catalogue with the the catalogue request with the filter expression shown in the example. The filter use the **dct type** to filter for the registry offer.

dDTR Asset

```
{
  "@context": {
    "dct": "http://purl.org/dc/terms/"
  },
  "protocol": "dataspace-protocol-http",
  "counterPartyAddress": "DATAPROVIDERURL/api/v1/dsp",
  "counterPartyId": "PARTNERURL",
  "querySpec": {
    "@type": "QuerySpecDto",
    "https://w3id.org/edc/v0.0.1/ns/offset": 0,
    "https://w3id.org/edc/v0.0.1/ns/limit": 10,
    "https://w3id.org/edc/v0.0.1/ns/filterExpression": {
      "@type": "CriterionDto",
      "operandLeft": "http://purl.org/dc/terms/type",
      "operator": "like",
      "operandRight": "%https://w3id.org/catenax/taxonomy#DigitalTwinRegistry%"
    }
  }
}
```

Register Digital Twins in DTR

Registering digital twins in the DTR is straight forward. For convenience for this example, **we register every digital twin in one step, i.e., including all submodel descriptors**. The DTR API also allows you to create the digital twin in multiple steps. Some additional explanatory notes:

- The property `idShort` of a digital twin entry **must be unique**. So we added a date counter (e.g., "Battery 2024-01-01-0001" instead of just "Battery").
- The visibility of certain attributes must be restricted. For example, **not all specific asset IDs** of a digital twin are allowed to be public readable. For this example, we still made all IDs public. Otherwise, we would have needed to add visibility permissions to the digital twin whenever a new partner wants to access the example. To limit visibility, you would use the **property externalSubjectId**. Here's an example that grants the Catena-X partner with BPN <PARTNER_BPN> access to the specific asset ID `partInstanceId`:

```

{
  "name": "partInstanceId",
  "value": "T588540140123B1011B018362515271",
  "externalSubjectId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "<PARTNER_BPN>"
      }
    ]
  }
}

```

More details about digital twin visibility can be found in the documentation of the Catena-X OpenSource DTR: <https://github.com/eclipse-tractusx/sldt-digital-twin-registry/blob/main/docs/architecture/6-crosscutting-concepts.md#access-control-to-digital-twins-based-on-the-bpn-business-partner-number-tenantid>.

Registering the Vehicle Digital Twin (Part Instance)

The following example shows the full digital for a Vehicle including submodel descriptors for SerialPart and SingleLevelBomAsBuilt submodels. This digital twin is registered using the method **POST /shell-descriptors** of the DTR API.

- Mandatory and optional specific asset IDs depend on the actual type of the part for which the digital twin is created: serial part, batch, or Just-In-Sequence part. More details can be found [here](#). Vehicles are serial parts in this context as they are identified using a serial number (VIN or - in pseudonymized form - VAN).
- For every submodel descriptor, the aspect model which defines the schema of the submodel must be specified using the semanticId property.
- Property subprotocolBody where the submodel data can be found: dspEndpoint points to the EDC (more accurately, to the catalog service of the EDC), id points to the EDC asset from which the submodel can be transferred.
- Once the data consumer, successfully negotiated a contract for this EDC asset, the URL in the href property must be used to actually transfer the data.
- Depending on the approach how submodels are published in the EDC (with or without asset bundling), these properties will look different. The sections below give more details about this.

AAS descriptor + Submodel descriptor

```

{
  "id": "urn:uuid:da071e28-8cf2-46a1-b5af-65231887c7c5",
  "globalAssetId": "urn:uuid:cdc8b050-ee17-432e-9ca6-b85d8017811d",
  "idShort": "BMW i4 2024-01-01-0001",
  "specificAssetIds": [
    {
      "name": "digitalTwinType",
      "value": "PartInstance",
      "externalSubjectId": {
        "type": "ExternalReference",
        "keys": [
          {
            "type": "GlobalReference",
            "value": "PUBLIC_READABLE"
          }
        ]
      }
    }
  ],
  {
    "name": "manufacturerId",
    "value": "BPNL0000000J8CWU",
    "externalSubjectId": {
      "type": "ExternalReference",
      "keys": [
        {
          "type": "GlobalReference",
          "value": "<PARTNER_BPN>"
        }
      ]
    }
  }
},
{

```



```

    "name": "manufacturerPartId",
    "value": "G26",
    "externalSubjectId": {
      "type": "ExternalReference",
      "keys": [
        {
          "type": "GlobalReference",
          "value": "PUBLIC_READABLE"
        }
      ]
    }
  },
  {
    "name": "partInstanceId",
    "value": "Skg0S0EzMjcwS0MwMdc00Tc=",
    "externalSubjectId": {
      "type": "ExternalReference",
      "keys": [
        {
          "type": "GlobalReference",
          "value": "<PARTNER_BPN>"
        }
      ]
    }
  },
  {
    "name": "intrinsicId",
    "value": "Skg0S0EzMjcwS0MwMdc00Tc=",
    "externalSubjectId": {
      "type": "ExternalReference",
      "keys": [
        {
          "type": "GlobalReference",
          "value": "<PARTNER_BPN>"
        }
      ]
    }
  },
  {
    "name": "van",
    "value": "Skg0S0EzMjcwS0MwMdc00Tc=",
    "externalSubjectId": {
      "type": "ExternalReference",
      "keys": [
        {
          "type": "GlobalReference",
          "value": "<PARTNER_BPN>"
        }
      ]
    }
  }
],
"submodelDescriptors": [
  {
    "endpoints": [
      {
        "interface": "SUBMODEL-3.0",
        "protocolInformation": {
          "href": "https://dataplane-partner.edc.aws.bmw.cloud/public/submodel",
          "endpointProtocol": "HTTP",
          "endpointProtocolVersion": [
            "1.1"
          ],
          "subprotocol": "DSP",
          "subprotocolBody": "id=Vehicle_2024-01-01-0001_Submodel_SerialPart;dspEndpoint=https://connector-partner.edc.aws.bmw.cloud/api/v1/dsp",
          "subprotocolBodyEncoding": "plain",
          "securityAttributes": [
            {
              "type": "NONE",
              "key": "NONE",

```

```

        "value": "NONE"
      }
    ]
  }
},
"idShort": "serialPart",
"id": "urn:uuid:9624aa4d-cfba-466c-b67a-6e7227e3fb54",
"semanticId": {
  "type": "ExternalReference",
  "keys": [
    {
      "type": "GlobalReference",
      "value": "urn:samm:io.catenax.serial_part:3.0.0#SerialPart"
    }
  ]
}
},
{
  "endpoints": [
    {
      "interface": "SUBMODEL-3.0",
      "protocolInformation": {
        "href": "https://dataplane-partner.edc.aws.bmw.cloud/public/submodel",
        "endpointProtocol": "HTTP",
        "endpointProtocolVersion": [
          "1.1"
        ],
        "subprotocol": "DSP",
        "subprotocolBody": "id=Vehicle_2024-01-01-0001_Submodel_SingleLevelBomAsBuilt; dspEndpoint=https://connector-partner.edc.aws.bmw.cloud/api/v1/dsp",
        "subprotocolBodyEncoding": "plain",
        "securityAttributes": [
          {
            "type": "NONE",
            "key": "NONE",
            "value": "NONE"
          }
        ]
      }
    }
  ],
  "idShort": "singleLevelBomAsBuilt",
  "id": "urn:uuid:b0a8dca2-34fe-41b4-a19e-710cd337b3b6",
  "semanticId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "urn:samm:io.catenax.single_level_bom_as_built:3.0.0#SingleLevelBomAsBuilt"
      }
    ]
  }
}
]
}
}

```

Digital Twin Type : Part Type

Info: Example of a battery cell **LI-ION ZELLE PHEV1 26AH MK02 C2**

AAS descriptor + Submodel descriptor

```

{
  "id": "urn:uuid:67b89e05-7506-4412-a929-1cd548f31861",
  "globalAssetId": "urn:uuid:5d63f272-8029-4ee6-8e3d-3b1831a30b36",
  "idShort": "Battery-Cell-Gen5",
  "specificAssetIds": [

```

```

{
  "name": "digitalTwinType",
  "value": "PartType",
  "externalSubjectId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "PUBLIC_READABLE"
      }
    ]
  }
},
{
  "name": "manufacturerId",
  "value": "BPNL0000000J8CWU",
  "externalSubjectId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "<PARTNER_BPN>"
      }
    ]
  }
},
{
  "name": "manufacturerPartId",
  "value": "188006",
  "externalSubjectId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "PUBLIC_READABLE"
      }
    ]
  }
},
{
  "name": "customerPartId",
  "value": "2373700",
  "externalSubjectId": {
    "type": "ExternalReference",
    "keys": [
      {
        "type": "GlobalReference",
        "value": "<PARTNER_BPN>"
      }
    ]
  }
}
],
"submodelDescriptors": [
  {
    "endpoints": [
      {
        "interface": "SUBMODEL-3.0",
        "protocolInformation": {
          "href": "https://dataplane-partner.edc.aws.bmw.cloud/public/submodel",
          "endpointProtocol": "HTTP",
          "endpointProtocolVersion": [
            "1.1"
          ],
          "subprotocol": "DSP",
          "subprotocolBody": "id=Submodel_Cell_SecondaryMaterialContent;dspEndpoint=https://connector-partner.edc.aws.bmw.cloud/api/v1/dsp",
          "subprotocolBodyEncoding": "plain",
          "securityAttributes": [
            {
              "type": "NONE",

```

```

        "key": "NONE",
        "value": "NONE"
    }
}
    ]
}
    ],
    "idShort": "secondaryMaterialContent",
    "id": "X",
    "semanticId": {
        "type": "ExternalReference",
        "keys": [
            {
                "type": "GlobalReference",
                "value": "urn:samm:io.catenax.secondary_material_content_calculated:1.0.0#SecondaryMaterialContent"
            }
        ]
    }
}
},
{
    "endpoints": [
        {
            "interface": "SUBMODEL-3.0",
            "protocolInformation": {
                "href": "https://dataplane-partner.edc.aws.bmw.cloud/public/submodel",
                "endpointProtocol": "HTTP",
                "endpointProtocolVersion": [
                    "1.1"
                ],
                "subprotocol": "DSP",
                "subprotocolBody": "id=Submodel_Cell_ProductCarbonFootprint;dspEndpoint=https://connector-partner.edc.aws.bmw.cloud/api/v1/dsp",
                "subprotocolBodyEncoding": "plain",
                "securityAttributes": [
                    {
                        "type": "NONE",
                        "key": "NONE",
                        "value": "NONE"
                    }
                ]
            }
        }
    ]
},
    "idShort": "productCarbonFootprint",
    "id": "urn:uuid:43c1965b-f86c-409d-a683-9d77ab68b4e8",
    "semanticId": {
        "type": "ExternalReference",
        "keys": [
            {
                "type": "GlobalReference",
                "value": "urn:samm:com.bmw.pcf:1.0.0#Pcf"
            }
        ]
    }
}
}
}
}
}

```

Setup Submodels of Digital Twins in EDC

For all assets, we use the same access and usage policies. In this example we use the same access policy like already created for the dDTR. Technically, we create four usage policies, one for SerialPart, one for SingleLevelBomAsBuilt, one for Secondary Material Content and one for Product Carbon Footprint, but the content (except their ID) is exactly the same. You can use different usage policies for submodels, but for SerialPart and SingleLevelBomAsBuilt this is not necessary. The usage purposes are standardized and are stored on the odr1 repo of the catena-x association project.

EDC Setup for Traceability

Usage Policy for SerialPart Submodels

```
{
  "@id": "usage-policy-for-aspect-model-serialpart",
  "@type": "PolicyDefinition",
  "profile": "cx-policy:profile2405",
  "policy": {
    "@type": "odrl:Set",
    "odrl:permission": {
      "odrl:action": {
        "@id": "odrl:use"
      },
      "odrl:constraint": {
        "odrl:and": [
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:FrameworkAgreement"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "DataExchangeGovernance:1.0"
          },
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:UsagePurpose"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "cx.core.industrycore:1"
          }
        ]
      }
    }
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

Contract Definition for All SerialPart Submodels

```
{
  "@id": "Contract-Definition-Vehicle-Submodel-SerialPart",
  "@type": "ContractDefinition",
  "accessPolicyId": "access-policy",
  "contractPolicyId": "usage-policy-for-aspect-model-serialpart",
  "assetsSelector": {
    "@type": "Criterion",
    "operandLeft": "https://admin-shell.io/aas/3/0/HasSemantics/semanticId",
    "operator": "like",
    "operandRight": "%urn:samm:io.catenax.serial_part:3.0.0#SerialPart%"
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

For the above contract definition, be use a filter that is based on one of the properties we use for submodel EDC assets. The filter selects all EDC assets that represent submodels of type SerialPart (aspect model with semanticId: urn:samm:io.catenax.serial_part:3.0.0#SerialPart). With this filter, the contract definition use automatically used for all EDC assets that are correctly created using this property.

EDC Setup for Sustainability & Circular Economy

For this example we create a specific usage policy for secondary material content and for product carbon footprint. For the this example we use the frameworkagreement **"DataExchangeGovernance:1.0"** and the usage purposes **"cx.circular.smc:1"** and **cx.pcf.base:1**. In the examples shows you to setup the usage policies for the specific data clusters pcf und smc.

EDC Setup Secondary Material Content

Usage Policy SMC

```
{
  "@id": "usage-policy-for-aspect-model-secondaryMaterialContent",
  "@type": "PolicyDefinition",
  "profile": "cx-policy:profile2405",
  "policy": {
    "@type": "odrl:Set",
    "odrl:permission": {
      "odrl:action": {
        "@id": "odrl:use"
      },
      "odrl:constraint": {
        "odrl:and": [
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:FrameworkAgreement"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "DataExchangeGovernance:1.0"
          },
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:UsagePurpose"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "cx.circular.smc:1"
          }
        ]
      }
    }
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

Contract definition SMC

```
{
  "@id": "Contract-Definition-BatteryCell-Submodel-SecondaryMaterialContent",
  "@type": "ContractDefinition",
  "accessPolicyId": "access-policy",
  "contractPolicyId": "usage-policy-for-aspect-model-productCarbonFootprint",
  "assetsSelector": {
    "@type": "Criterion",
    "operandLeft": "https://admin-shell.io/aas/3/0/HasSemantics/semanticId",
    "operator": "like",
    "operandRight": "%urn:samm:com.bmw.pcf:1.0.0#Pcf%"
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

EDC Setup Product Carbon Footprint

Usage Policy SMC

```
{
  "@id": "usage-policy-for-aspect-model-productCarbonFootprint",
  "@type": "PolicyDefinition",
  "profile": "cx-policy:profile2405",
  "policy": {
    "@type": "odrl:Set",
    "odrl:permission": {
      "odrl:action": {
        "@id": "odrl:use"
      },
      "odrl:constraint": {
        "odrl:and": [
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:FrameworkAgreement"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "DataExchangeGovernance:1.0"
          },
          {
            "odrl:leftOperand": {
              "@id": "cx-policy:UsagePurpose"
            },
            "odrl:operator": {
              "@id": "odrl:eq"
            },
            "odrl:rightOperand": "cx.pcf.base:1"
          }
        ]
      }
    }
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```

Contract definition SMC

```
{
  "@id": "Contract-Definition-BatteryCell-Submodel-productCarbonFootprint",
  "@type": "ContractDefinition",
  "accessPolicyId": "access-policy",
  "contractPolicyId": "usage-policy-for-aspect-model-productCarbonFootprint",
  "assetsSelector": {
    "@type": "Criterion",
    "operandLeft": "https://admin-shell.io/aas/3/0/HasSemantics/semanticId",
    "operator": "like",
    "operandRight": "%urn:samm:com.bmw.pcf:1.0.0#Pcf%"
  },
  "@context": {
    "@vocab": "https://w3id.org/edc/v0.0.1/ns/",
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "tx": "https://w3id.org/tractusx/v0.0.1/ns/",
    "tx-auth": "https://w3id.org/tractusx/auth/",
    "cx-policy": "https://w3id.org/catenax/policy/",
    "odrl": "http://www.w3.org/ns/odrl/2/"
  }
}
```



Currently the guidance how to deal with pcf data that relates to the EU Batterypass regulatory is not defined in CX. Based on this this pcf value do not relates to the pcf data model, rulebook and processes which are established in Catena-X. This example focus on how to handle digital twins in catena-x. the pcf discussion is placed and still ongoing! For this example BMW creates a own data model **urn:samm:com.bmw.pcf:1.0.0**

Publish Submodels without Asset Bundling (One EDC Asset per Submodel)

For the Vehicle digital twin, we chose an approach without asset bundling, i.e., we create an EDC asset for every submodel of this digital twin. This approach is described as [Option 1 in the Industry Core KIT](#). The Vehicle digital twin "Vehicle 2024-01-01-0001" has two submodels, so we need to create two EDC assets with policies. The necessary steps are the same for both submodels:

- Create the asset for the submodel (SerialPart, SingleLevelBomAsBuilt)
- If not yet created, create access and usage policies for the asset. As these policies might be shared between assets, they only need to be created once.
- Create the contract definition for the asset
- After this step, the EDC asset for the submodel is visible in the EDC catalog for every partner that has access based on the configured access policy in the contract definition.

Asset for Submodel SerialPart for the Vehicle Digital Twin

```
{
  "@context": {
    "dct": "http://purl.org/dc/terms/",
    "cx-common": "https://w3id.org/catenax/ontology/common#",
    "cx-taxo": "https://w3id.org/catenax/taxonomy#",
    "aas-semantic": "https://admin-shell.io/aas/3.0/HasSemantics/"
  },
  "@type": "Asset",
  "@id": "Vehicle_2024-01-01-0001_Submodel_SerialPart",
  "properties": {
    "dct:type": {
      "@id": "cx-taxo:Submodel"
    },
    "cx-common:version": "3.0",
    "aas-semantic:semanticId": {
      "@id": "urn:samm:io.catenax.serial_part:3.0.0#SerialPart"
    }
  },
  "dataAddress": {
    "@type": "DataAddress",
    "secretAccessKey": "...",
    "accessKeyId": "...",
    "region": "eu-west-1",
    "type": "AmazonS3",
    "bucketName": "digital-twin-exchange-example",
    "objectName": "Vehicle_2024-01-01-0001_Submodel_SerialPart.json"
  }
}
```

The asset - via its dataAddress - points to an S3 bucket where the actual submodel is stored as a JSON file. The JSON file for the Vehicle SerialPart submodel looks like this:

Submodel SerialPart for Vehicle Digital Twin

```
{
  "catenaXId": "urn:uuid:cdc8b050-ee17-432e-9ca6-b85d8017811d",
  "localIdentifiers": [
    {
      "key": "manufacturerId",
      "value": "BPNL0000000000ISY"
    },
    {
      "key": "partInstanceId",
      "value": "NzFhZmFkNmItMGU5Ny00NzNkLWFIZDItoWY5NWU2NzE0OTVi"
    },
    {
      "key": "van",
      "value": "NzFhZmFkNmItMGU5Ny00NzNkLWFIZDItoWY5NWU2NzE0OTVi"
    }
  ],
  "manufacturingInformation": {
    "date": "2024-01-01T00:00:00.000Z",
    "country": "DEU"
  },
  "partTypeInformation": {
    "manufacturerPartId": "G60",
    "classification": "product",
    "nameAtManufacturer": "Vehicle"
  }
}
```

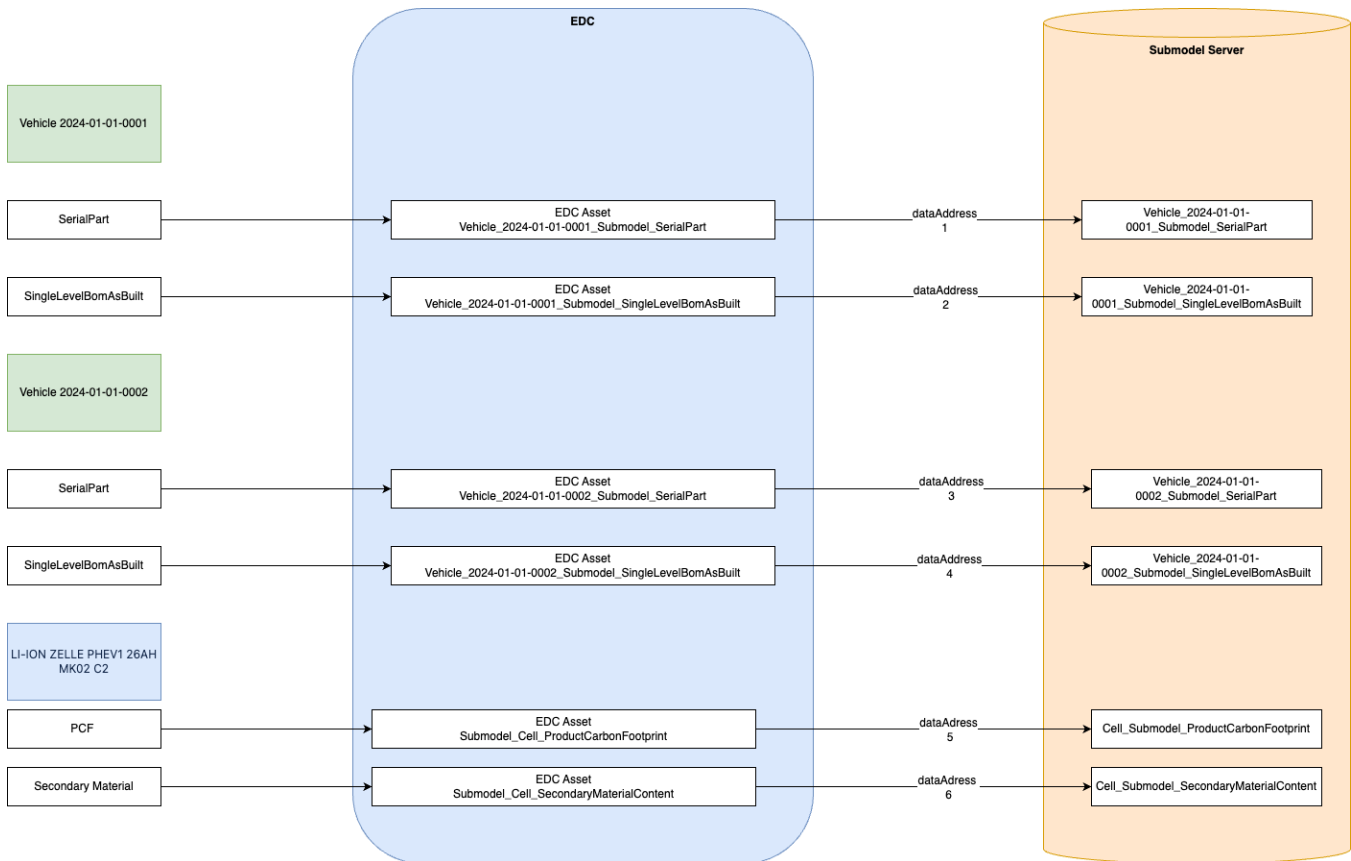
The digital twins for Vehicle do not use asset bundling, but one asset is created for every submodel of a Vehicle digital twin:

- The actual submodel is represented by the asset in the EDC
- href is only the URL to the EDC data plane

? Use the same usage policy for all serialized submodels (SerialPart and SingleLevelBomAsBuilt)?

? Use the same usage policy for all type based submodels

On a more general level, when you create several Vehicle digital twins, the structure should look like in the following image:



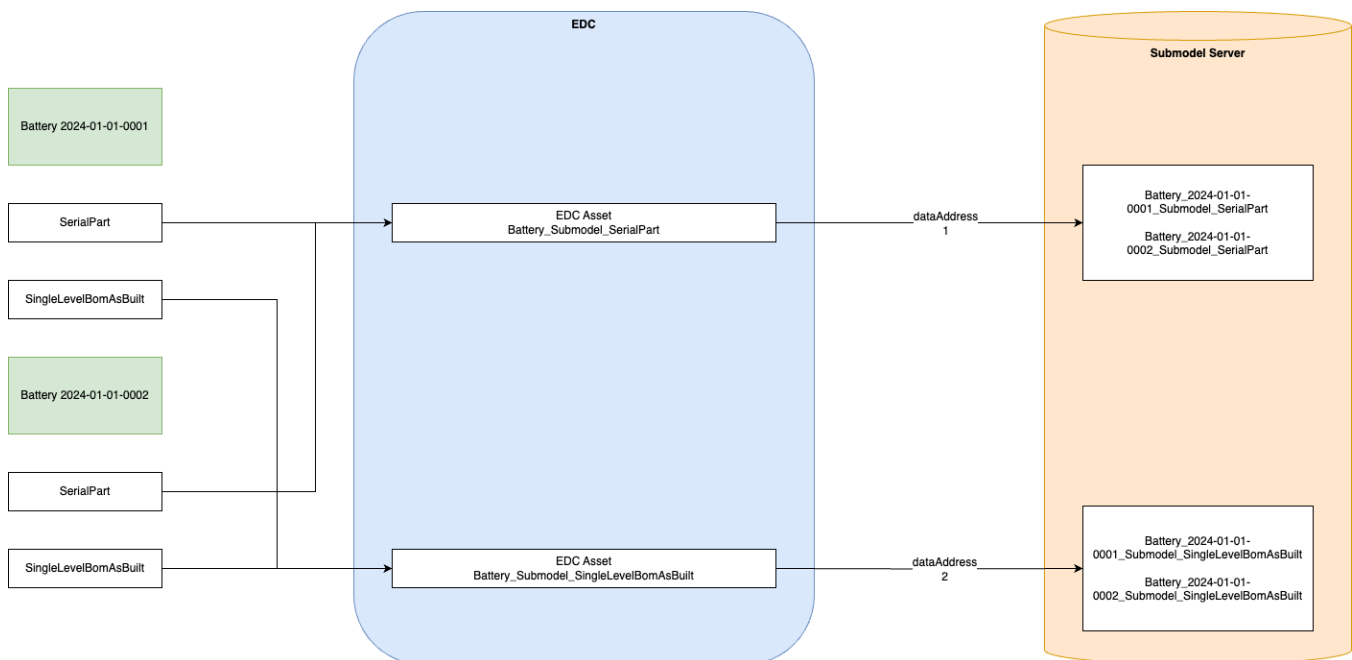
Publish Submodels with Asset Bundling (One EDC Asset per Aspect Model)

For the Battery digital twin, we chose an approach with asset bundling, i.e., we create an EDC asset for every aspect model that is used to transfer all submodels of this aspect model for *all* Battery digital twin. This approach is described as [Option 2 in the Industry Core KIT](#). The Battery digital twin "Battery 2024-01-01-0001" only has one submodel, so we only need to create one EDC assets with policies.

- Create the asset for the aspect model of Battery digital twins (SerialPart)
- If not yet created, create access and usage policies for the asset. As these policies might be shared between assets, they only need to be created once.
- Create the contract definition for the asset
- After this step, the EDC asset for SerialPart submodels of Battery digital twins is visible in the EDC catalog for every partner that has access based on the configured access policy in the contract definition.

Asset for Submodel SerialPart for the Battery Digital Twin

```
{
  "@context": {
    "dct": "http://purl.org/dc/terms/",
    "cx-common": "https://w3id.org/catenax/ontology/common#",
    "cx-taxo": "https://w3id.org/catenax/taxonomy#",
    "aas-semantic": "https://admin-shell.io/aas/3/0/HasSemantics/"
  },
  "@type": "Asset",
  "@id": "Battery_Submodel_SerialPart",
  "properties": {
    "dct:type": {
      "@id": "cx-taxo:Submodel"
    },
    "cx-common:version": "3.0",
    "aas-semantic:semanticId": {
      "@id": "urn:samm:io.catenax.serial_part:3.0.0#SerialPart"
    }
  },
  "dataAddress": {
    "@type": "DataAddress",
    "type": "HttpData",
    "baseUrl": "https://c132ew588i.execute-api.eu-west-1.amazonaws.com/dev/digital-twin-exchange-example",
    "proxyQueryParams": "false",
    "proxyPath": "true",
    "proxyMethod": "false",
    "authKey": "x-api-key",
    "authCode": "..."
  }
}
```



The asset bundling used in this example is a simplified version. With asset bundling, you need to ensure in the Submodel Server, that the data consumer, that transfers the data via the EDC, is actually allowed to see and transfer the submodel for a digital twin, if there are more than one partner allowed to fetch, e.g. Battery submodels. One way to solve this is to create separate EDC assets per supplier (and per submodel), so, e.g. Battery_Submodel_SerialPart_Partner_A and Battery_Submodel_SerialPart_Partner_B.

Create Submodels in the Submodel Server

For this example, we use a very simple Submodel Server. Basically, the Submodel Server is referenced in the dataAddress property of the EDC asset description. The actual properties and values depend on the Submodel Server you use. So, this documentation can only serve as an example.

For non-asset-bundling submodels, we use a simple S3 bucket as Submodel Server:

digital-twin-exchange-example [Info](#)

[Objekte](#) | [Eigenschaften](#) | [Berechtigungen](#) | [Metriken](#) | [Verwaltung](#) | [Access Points](#)

Objekte (4) [Info](#)

Aktionen

Ordner erstellen

Hochladen

Objekte sind die grundlegenden Entitäten, die in Amazon S3 gespeichert sind. Sie können [Amazon S3 Inventory](#) verwenden, um eine Liste aller Objekte in Ihrem Bucket abzurufen. Damit andere auf Ihre Objekte zugreifen können, müssen Sie ihnen explizit Berechtigungen erteilen. [Weitere Informationen](#)

Objekte nach Präfix suchen

< 1 >

☐	Name	▲ Typ ▼	Letzte Änderung ▼	Größe ▼	Speicherklasse
☐	Transmission_2024-01-01-0001_Submodel_JustInSequencePart.json	json	25.11.2024 12:11:52 PM CET	604.0 B	Standard
☐	urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22	-	25.11.2024 09:57:50 AM CET	580.0 B	Standard
☐	Vehicle_2024-01-01-0001_Submodel_SerialPart.json	json	25.11.2024 09:03:44 AM CET	531.0 B	Standard
☐	Vehicle_2024-01-01-0001_Submodel_SingleLevelBomAsBuilt.json	json	25.11.2024 09:03:44 AM CET	756.0 B	Standard

The dataAddress in the EDC asset directly points to the submodel (JSON file) in the S3 bucket that will be transferred when a data consumer requests this submodel:

```
"dataAddress": {
  "@type": "DataAddress",
  "secretAccessKey": "...",
  "accessKeyId": "...",
  "region": "eu-west-1",
  "type": "AmazonS3",
  "bucketName": "digital-twin-exchange-example",
  "objectName": "Vehicle_2024-01-01-0001_Submodel_SerialPart.json"
}
```

Without asset bundling, every submodel points to a unique EDC asset which references a unique JSON file in the S3 bucket. So, it's easily extendible with further submodels for Vehicle digital twins, e.g., with Vehicle_2024-01-01-0002_Submodel_SerialPart.json, Vehicle_2024-01-01-0003_Submodel_SerialPart.json, and so on.

For asset-bundling submodels, we cannot do that as the same EDC asset must be able to return not just one fixed submodel, but a submodel that depends on the input request to the EDC / Submodel Server. Therefore, we use a (simple) REST API to serve these requests. The dataAddress in this case looks like this:

```
"dataAddress": {
  "@type": "DataAddress",
  "type": "HttpData",
  "baseUrl": "https://c132ew588i.execute-api.eu-west-1.amazonaws.com/dev/digital-twin-exchange-example",
  "proxyQueryParams": "false",
  "proxyPath": "true",
  "proxyMethod": "false",
  "authKey": "x-api-key",
  "authCode": "..."
}
```

After a successful contract negotiation, a data consumer invokes the data plane of the EDC with the URL specified in the href property of the submodel descriptor. For the Battery digital twin, the href looks like this:

```
https://dataplane-partner.edc.aws.bmw.cloud/public/urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22/submodel
```

Note that the data consumer must appen "\$value" to this URL as defined in the [Digital Twin KIT](#).

The EDC will transform this URL given the information in the asset's dataAddress by replacing the EDC dataplane with the basePath from the dataAddress.

```
https://c132ew588i.execute-api.eu-west-1.amazonaws.com/dev/digital-twin-exchange-example/urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22/submodel/$value
```

Then, the EDC retrieves the data from this URL using a GET request and returns the data.

We implemented a simple REST API that retrieves the submodel specified in the URL (urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22) from the above S3 bucket.

The difference to non-asset bundling can be seen when comparing both href URLs:

- Without asset-bundling: [https://dataplane-partner.edc.aws.bmw.cloud/public/submodel/\\$value](https://dataplane-partner.edc.aws.bmw.cloud/public/submodel/$value)
 - Here, the EDC asset directly points to the submodel JSON file (via its dataAddress), so no further information besides the dataplane is needed in the href URL.
- With asset-bundling: [https://dataplane-partner.edc.aws.bmw.cloud/public/urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22/submodel/\\$value](https://dataplane-partner.edc.aws.bmw.cloud/public/urn:uuid:aa20059b-2dfc-49fc-a16d-fd2e088f0c22/submodel/$value)
 - Here, the EDC asset is shared between all submodels of the SerialPart aspect model for Battery digital twins. Without a URL like the one without asset bundling, the Submodel Server (i.e., the REST API) would not know which of all available submodels it should return. Therefore, the href URL must specify which submodel actually to fetch based on the submodel descriptor (and therefore digital twin) in which its defined.
 - For the example, we use the submodel ID as ID as it's must be unique anyway. The REST API Submodel Server uses this submodel ID and looks for the corresponding file in the S3 bucket shown above.

Use Full Definition

Aspect	Aspects are services that provide data about assets represented by Digital Twins. Each aspect exposes an endpoint for supported data transport protocols (HTTP or MQTT -> Not used at the moment in Catena-X). When a data consumer, typically a service that intends to use the data, locates a relevant Twin in the Digital Twin Registry, it can obtain two key pieces of information from that Twin: 1. The endpoint of the aspect 2. The identifier of the Aspect Model corresponding to the aspect's runtime data The data consumer can then retrieve data from the aspect as needed. The aspect implementer is responsible for ensuring the runtime data sent by the aspect aligns with the respective Aspect Model. Aspect data consumers can be implemented in two ways: 1. Specifically for a particular aspect type, by referencing the corresponding Aspect Model during development or generating client/deserialization code from it. 2. Generically, by interpreting the Aspect Model at runtime and using that to deserialize the data.	N/A
--------	--	-----

Aspect Model	An Aspect Model is a formal (machine-readable) description of the structure and semantics of the data provided by an Aspect. The Aspect Model serves two key purposes: 1. Capturing Domain Semantics: • The Aspect Model captures the domain-specific semantics of the part of the Digital Twin it describes, acting as an ontology. • It makes explicit and consistent the information that is often available informally (e.g., in a data sheet) or implicitly (as tacit knowledge) across a broader scope than a single service or application. 2. Defining a Contract: • The Aspect Model serves as a contract between the data provider and data consumer, similar to a schema description (e.g., JSON Schema, XML Schema). • It precisely defines the data structures and values that may appear in the runtime data. • Unlike a pure schema language, the Aspect Model also contains the previously mentioned domain semantics, which can be used for data validation beyond just the structural constraints. In summary, the Aspect Model bridges the gap between the domain-specific knowledge and the technical representation of data, providing a formal, machine-readable way to describe and validate the information contained within a Digital Twin's Aspect.	Aspect Model: <pre> :BatteryPass a samm:Aspect ; samm:preferredName "Battery Passport"@en ; samm:description "The battery pass describes information collected during the lifecycle of a battery." ; samm:see <https://environment.ec.europa.eu/publications/proposal-ecodesign-sustainable-products-regulation> ; samm:see <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32023R1542&qid=1693215264963> ; samm:properties (:identification :operation :characteristics :sustainability :materials :performance) ; samm:operations () ; samm:events () . :identification a samm:Property ; samm:preferredName "Identification"@en ; samm:description "Battery identification involves specific battery identifiers, including those assigned to the battery components." ; samm:characteristic :IdentificationCharacteristic . :operation a samm:Property ; samm:preferredName "Operation"@en ; samm:description "General operation information includes details such as the manufacturer's identification." ; samm:characteristic :OperationCharacteristic . </pre> Aspect Model Data: <pre> 1 { 2 "characteristics": { 3 "physicalDimension": { 4 "length": { 5 "value": 20.0, 6 "unit": "unit:millimetre" 7 }, 8 "width": { 9 "value": 20.0, 10 "unit": "unit:millimetre" 11 }, 12 "weight": { 13 "value": 20.0, 14 "unit": "unit:gram" 15 }, 16 "height": { 17 "value": 20.0, 18 "unit": "unit:millimetre" 19 } 20 }, 21 "warranty": { 22 "lifetime": 36, 23 "lifetimeUnit": "unit:day" 24 } 25 }, 26 "metadata": { 27 "backupReference": "https://dummy.link", 28 "registrationIdentifier": "https://dummy.link/ID8283746239078", 29 "economicOperatorId": "BPMLE123456789ZZ", 30 "lastModification": "2000-01-01", 31 "predecessor": "urn:uuid:00000000-0000-0000-0000-000000000000", 32 "issueDate": "2000-01-01", 33 "version": "1.0.0", 34 "passportIdentifier": "urn:uuid:550e8400-e29b-41d4-a716-446655440000", 35 "status": "draft", 36 "expirationDate": "2030-01-01" 37 }, 38 "commercial": { 39 "placedOnMarket": "2000-01-01", 40 "purpose": ["automotive"] 41 }, 42 "identification": { 43 "chemistry": "Nickel Cobalt Manganese (NCM)", 44 "idRec": "34567890", 45 "identification": { 46 "batch": [{ 47 "value": "B1012345678", 48 "key": "batchId" 49 }], 50 "codes": [{ 51 "value": "8703 24 10 00", 52 "key": "TARIC" 53 }], 54 "type": { 55 "manufacturerPartId": "123-0.740-3434-A", 56 "nameOfManufacturer": "Mirror Left" 57 } 58 } 59 } 60 } </pre>
--------------	---	--

Asset Administration Shell	The Asset Administration Shell is a standardized framework that enables cross-vendor interoperability and supports the digital twin concept for Industry 4.0. Key features: • Enables standardized data exchange across product life cycles and production processes • Applicable to both intelligent and non-intelligent products • Provides a digital and interoperable foundation for autonomous systems and AI • Structures product data into standardized submodels for efficient processing and access • Supports the mapping of entire product life cycles and continuous value chains • Can present implementation challenges, especially for SMEs, that require guidance and support	N/A
Digital Twins	A Digital Twin is a digital representation of a physical asset, such as a machine, vehicle, or product. It can also represent a virtual or logical entity, like a machine type, product type, server, or fact sheet. There are multiple ways to implement the concept of a Digital Twin in practice. These include: 1. Representing a physical asset: The Digital Twin can mirror a real-world physical object, capturing its state, properties, and behavior. 2. Representing a virtual/logical entity: The Digital Twin can model a virtual or logical thing, such as a product type or server configuration. 3. Hybrid approach: The Digital Twin can combine elements of both physical and virtual/logical representations. The specific implementation approach depends on the use case and the nature of the asset or entity being modeled. Regardless of the approach, the core idea of a Digital Twin is to create a digital counterpart that can be used to simulate, analyze, and optimize the physical or virtual original.	<pre>graph TD; DT[Digital Twin e.g Battery Cell Typ] -.-> PCF[PCF]; DT -.-> MC[Material Comp.]; DT -.-> SRQ[SRQ]; DT -.-> Dots[...]; PCF --- Aspect; MC --- Aspect; SRQ --- Aspect; Dots --- Aspect; subgraph Aspect; PCF; MC; SRQ; Dots; end; DT --- DT_Label[Digital Twin];</pre>
Submodel	A submodel is used to structure the information model of an AAS (Asset Administration Shell) representing the technical functionality of the underlying asset into distinguishable parts. Each submodel refers to a well-defined domain or subject matter. Submodels can become standardized and thus become submodel templates. Submodels can have different life-cycles.	See Digital Twin Registry

Di
git
al
Tw
in
Re
gis
try

The Digital Twin Registry (DTR) serves as the central point of information about the existence of:

- Digital Twins and their associated Aspects
- Asset Administration Shells and their Submodels

The Digital Twin Registry is the primary means to locate and interact with digital twins. To enable this, the Digital Twin Registry provides several APIs.

Through these interfaces applications can discover, access, and work with the digital twins and their related components stored in the registry. The Digital Twin Registry acts as a hub, allowing users to find the necessary information and entry points to engage with the digital representations of physical or virtual assets.

